

Tecnológico de Monterrey

Evidencia 2: Estructuras lineales y aplicación de logs

Documento de evidencia

Diego Contreras Alvarez, A00845716

Programación de Estructuras de Datos y Algoritmos Fundamentales (TC1031)

Grupo 607

Profesor: David Alonso Cantú Delgado

Fecha: 18 de octubre de 2026

Nota sobre los archivos de entrada

Las instrucciones hablan de log608-1.txt y log608-2.txt, pero los archivos que se entregaron se llaman log607-1.txt y log607-2.txt, así que en este documento se usan con ese nombre. Además la descripción de las instrucciones viene al revés. Revisando los archivos se ve que log607-1.txt es el que está en orden aleatorio (empieza con Sep 29 2024, luego Feb 13 2026 y luego May 04 2025) y log607-2.txt es el casi ordenado (empieza con Sep 08 2024 a las 00:22, 04:37 y 05:50). Todo lo que sigue usa esa descripción real. El archivo de salida sí conserva el nombre output608.out que piden las instrucciones.

1. Evidencia de ejecución

1.1 Compilación

La aplicación y el programa de pruebas se compilan por separado, porque cada uno tiene su propio main. Los dos compilan sin advertencias con -Wall -Wextra.



Figura 1. Compilación de la aplicación y del programa de pruebas sin advertencias.

1.2 Programa de pruebas de la Parte A

El programa de pruebas (src/pruebas.cpp) revisa las cuatro estructuras en el mismo orden: vacía, un solo elemento, varias inserciones y eliminaciones en los extremos, e impresión. Cada revisión escribe [OK] o [FALLA] y al final se muestra el total. La corrida completa hace 140 revisiones con 0 fallas, y el texto completo de la salida está en docs/salida_pruebas.txt.

En la LinkedList se prueba que sacar de una lista vacía no cierre el programa y que tail se reinicie bien después de vaciarla. En la Pila se ve el orden LIFO, porque después de push(1, 2, 3) el primer pop saca el 3. En la Fila se ve el orden FIFO, porque después de enqueue(1, 2, 3) el primer dequeue saca el 1. En la lista doblemente ligada, después de cada cambio se recorre hacia adelante con next y hacia atrás con prev, y se revisa que los dos recorridos salgan al revés uno del otro. Si algún prev quedara mal, solo se notaría en el recorrido hacia atrás.

```
diegocontreras@Diegos-MacBook-Pro-101 Evidence2 % g++ -std=c++20 -O2 -Wall
g++ -std=c++20 -Wall -Wextra -Iinclude src/pruebas.cpp -o build/pruebas
diegocontreras@Diegos-MacBook-Pro-101 Evidence2 % ./build/pruebas
Pruebas de estructuras - Evidencia 2
Diego Contreras Alvarez, A00845716

==================== LinkedList ====================

Prueba 1. Lista vacia
  [OK] isEmpty() es verdadero
  [OK] size() = 0
  removeFront: la lista esta vacia, no hay nada que quitar
  [OK] removeFront sobre vacia regresa false y no cambia el valor
  [OK] getFront sobre vacia regresa false

Prueba 2. Un solo elemento
  [OK] insertBack(10) en vacia deja size() = 1
  [OK] el primero es 10
  [OK] removeFront saca el 10
  [OK] al quitar el unico nodo la lista queda vacia
  [OK] insertFront(20) en vacia funciona despues de vaciarla

Prueba 3. Insertar y quitar varias veces en los extremos
  [OK] insertFront(0,1) e insertBack(2,3) dejan 4 elementos
  Lista: 0-1-2-3
  [OK] removeFront saca el 0
  [OK] insertBack(4) y removeFront saca el 1
  [OK] se sacan 2, 3 y 4 y queda vacia
  removeFront: la lista esta vacia, no hay nada que quitar
  [OK] insertBack(7) despues de vaciarla (tail se reinicio bien)

Prueba 4. Imprimir
  Lista vacia:
  Lista 5, 6, 7: 5-6-7

==================== Stack ====================

Prueba 5. Pila vacia
  [OK] isEmpty() es verdadero
  [OK] size() = 0
  pop: la pila esta vacia
  [OK] pop sobre vacia regresa false y no cambia el valor
  peek: la pila esta vacia
  [OK] peek sobre vacia regresa false y no cambia el valor

Prueba 6. Un solo elemento
  [OK] push(10) deja size() = 1
  [OK] peek ve el 10
  [OK] peek no saca el elemento
  [OK] pop saca el 10
  [OK] la pila queda vacia

Prueba 7. Insertar y quitar varias veces en el extremo (arriba)
  [OK] push(1,2,3) deja el 3 arriba
  [OK] pop saca el 3 (el ultimo que entro)
  [OK] push(4) y pop saca el 4
  [OK] pop saca el 2
  [OK] pop saca el 1 y queda vacia
  pop: la pila esta vacia

Prueba 8. Imprimir (de arriba hacia abajo)
  Pila vacia:
  Despues de push 5, 6, 7: 7-6-5
```

Figura 2. LinkedList (pruebas 1 a 4) y Stack con orden LIFO (pruebas 5 a 8).



Figura 3. Queue con orden FIFO (pruebas 9 a 12) e inicio de DoublyLinkedList.



Figura 4. DoublyLinkedList: un solo elemento e insertAfter sobre el tail, en medio y en el head.



Figura 5. DoublyLinkedList: inserciones en los extremos y remove del head, del tail y de un nodo en medio.



Figura 6. DoublyLinkedList: quitar todo hasta el único nodo, impresión en los dos sentidos y resumen con 0 fallas.

1.3 Opciones del menú

Las corridas de esta sección se hicieron primero con log607-2.txt (casi ordenado) y al final con log607-1.txt (aleatorio), para que output608.out quedara generado con log607-1.txt.

Carga y opción 1 con log607-2.txt. Al iniciar, el programa pregunta qué archivo cargar, encola cada línea en la Fila, la procesa y la inserta al final de la lista doble. Reporta 6818 registros cargados y 0 líneas omitidas. Antes de ordenar pide la predicción del crecimiento de las comparaciones.

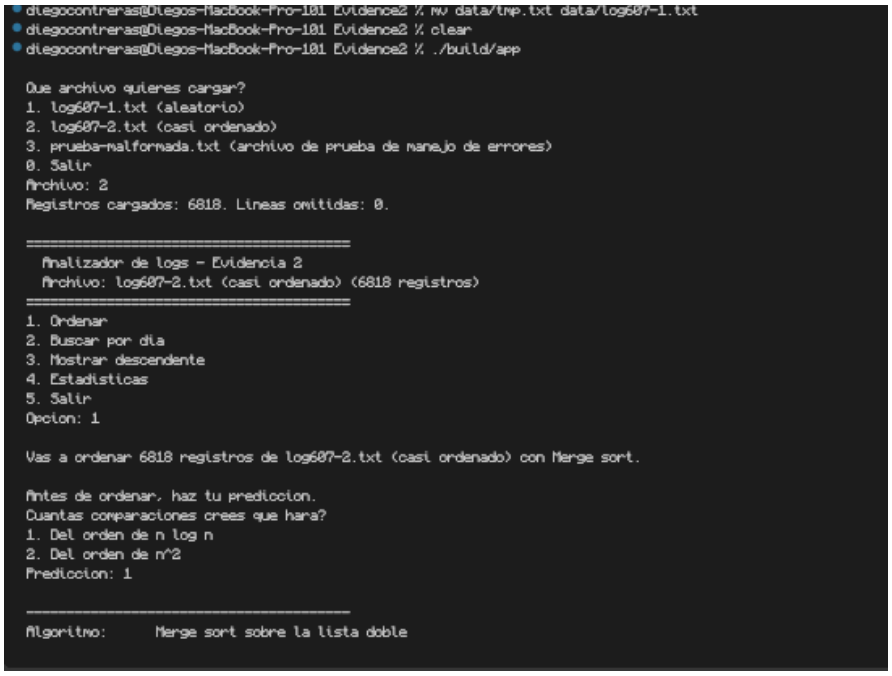


Figura 7. Carga de log607-2.txt y predicción antes de ordenar.

Opciones 1, 2 y 3. El resultado del ordenamiento muestra el algoritmo, el archivo, n, el tiempo, las comparaciones, la profundidad máxima y si coincidió la predicción. Al pedir otra vez la opción 1 el programa avisa que la lista ya está ordenada. La opción 2 busca Dec 16 2025 y encuentra 13 registros, y la opción 3 muestra esos mismos 13 registros del más reciente al más antiguo usando la Pila.



Figura 8. Resultado del ordenamiento de log607-2.txt, lista ya ordenada, búsqueda por día con formatos inválidos y vista descendente.

Opción 4. Las estadísticas se calculan con un solo recorrido de la lista. El día con más incidentes es Jan 27 2025 con 21 registros, el evento más frecuente es Malware Detection Alerts con 342, y para la IP hay un empate de 13 direcciones con 3 apariciones cada una, que se muestran todas.

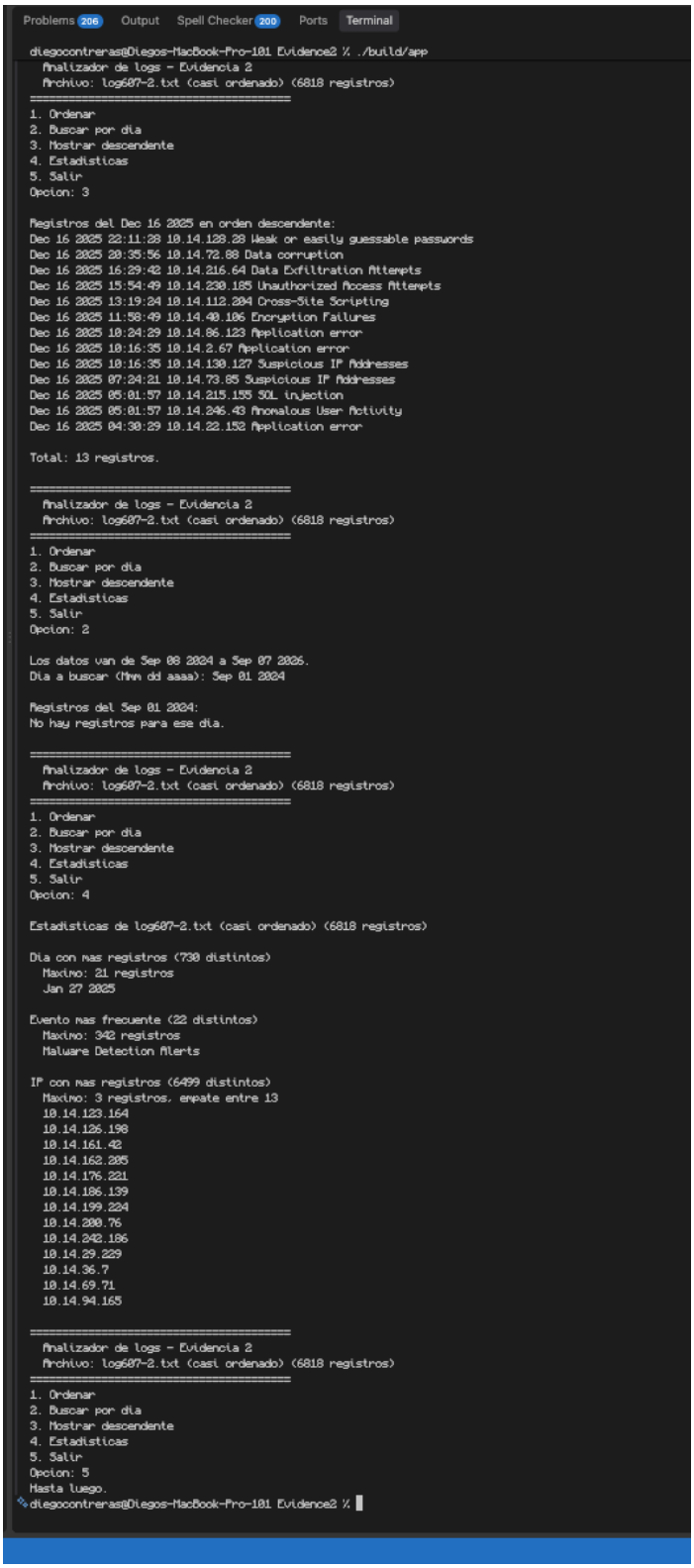


Figura 9. Vista descendente, día sin registros, estadísticas con empate y salida con la opción 5.

Opción 1 con log607-1.txt. Esta es la última corrida, la que deja generado el archivo de salida.

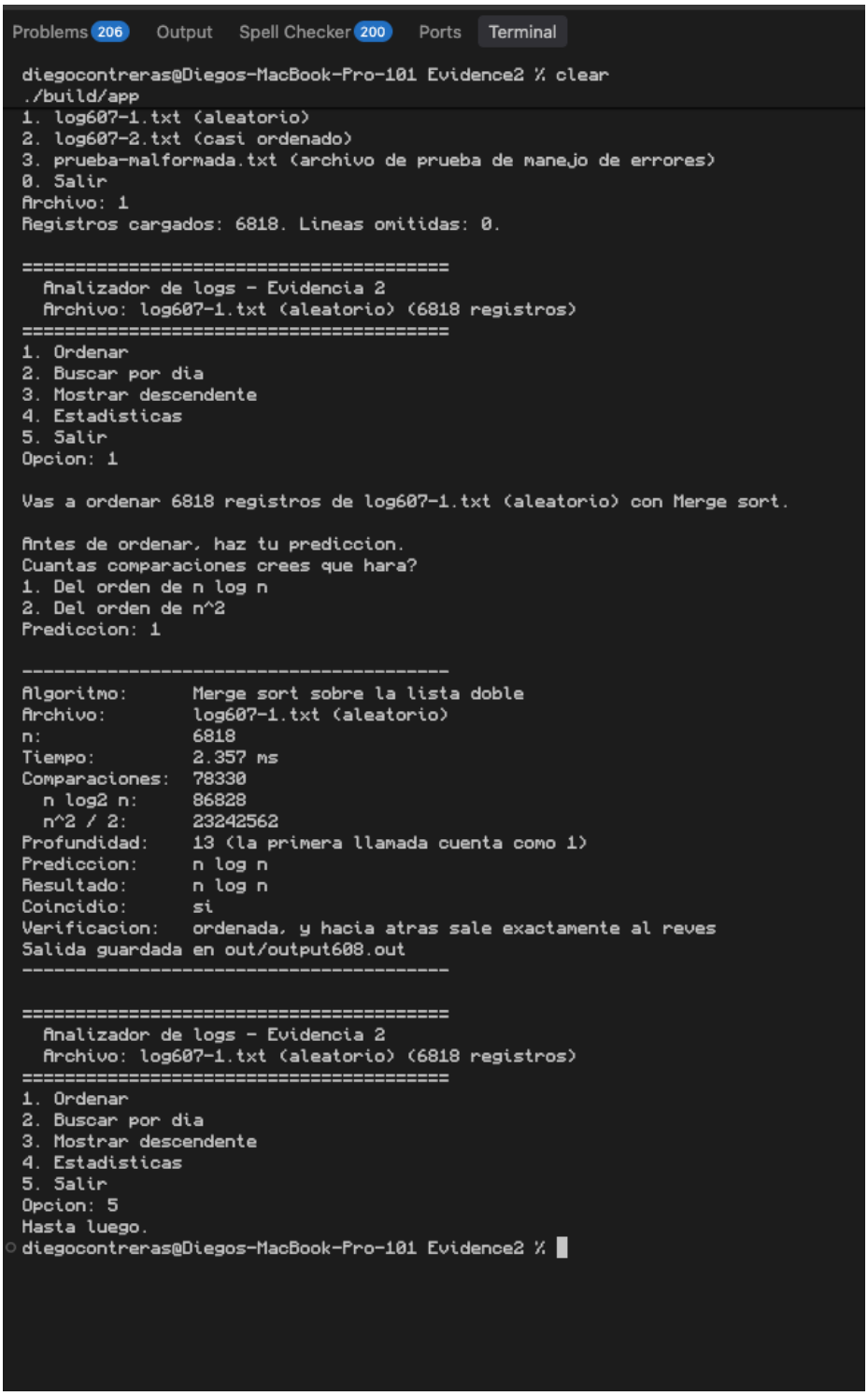


Figura 10. Carga y ordenamiento de log607-1.txt.

Parte del archivo output608.out. Se muestran las primeras y las últimas cinco líneas. El archivo tiene las 6818 líneas, empieza en el registro más antiguo (Sep 08 2024 00:22:43) y termina en el más reciente (Sep 07 2026 23:50:40), con el mismo formato que el archivo de entrada.

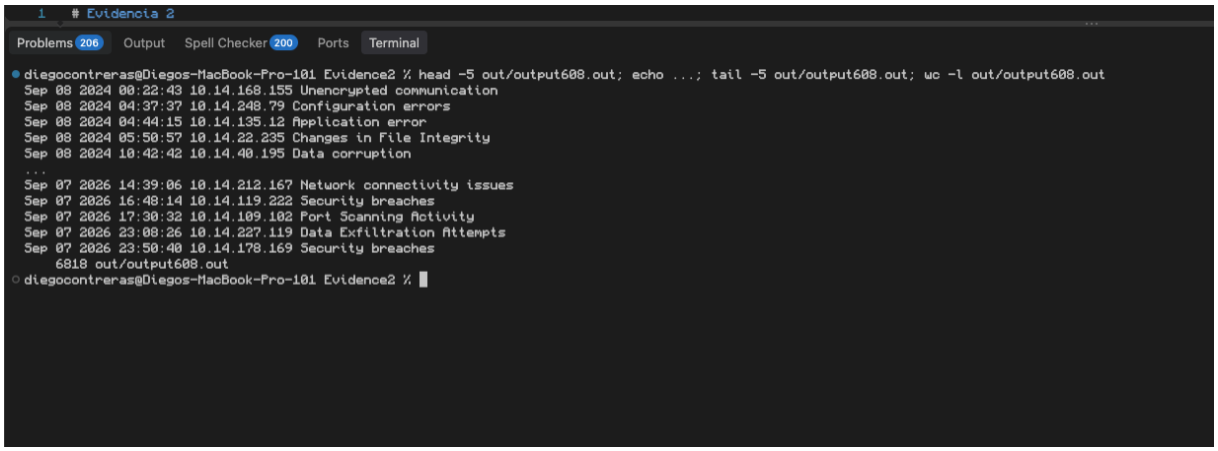


Figura 11. Primeras y últimas líneas de out/output608.out y su número de líneas.

1.4 Casos de borde

Caso	Entrada	Resultado obtenido	Figura
Archivo inexistente	log607-1.txt renombrado a tmp.txt, archivo 1	Muestra la ruta que no pudo abrir y vuelve a preguntar qué archivo cargar sin cerrarse	Figura 13
Línea con formato inválido	prueba-malformada.txt	Se omiten 4 de 6 líneas con su número y motivo: línea vacía, mes inválido (dos) y fecha imposible (Feb 30). Se cargan 2 registros	Figura 12
Día sin registros	Sep 01 2024	No hay registros para ese dia	Figura 9
Formato de día inválido	Feb 30 2025, Xyz 01 2025, Dec 16	Los tres se rechazan con el formato esperado y vuelve a pedir el día	Figura 8
Buscar antes de ordenar	Opción 2 sin ordenar	Primero ordena la lista con la opcion 1	Figura 12
Ordenar una lista ya ordenada	Opción 1 dos veces	La lista ya esta ordenada, no hay nada que hacer	Figura 8
Empates en las estadísticas	Opción 4	Muestra las 13 IPs empatadas con 3 apariciones	Figura 9
Opción de menú inexistente	9 y abc	Entrada invalida. Escribe un numero entre 1 y 5	Figura 12

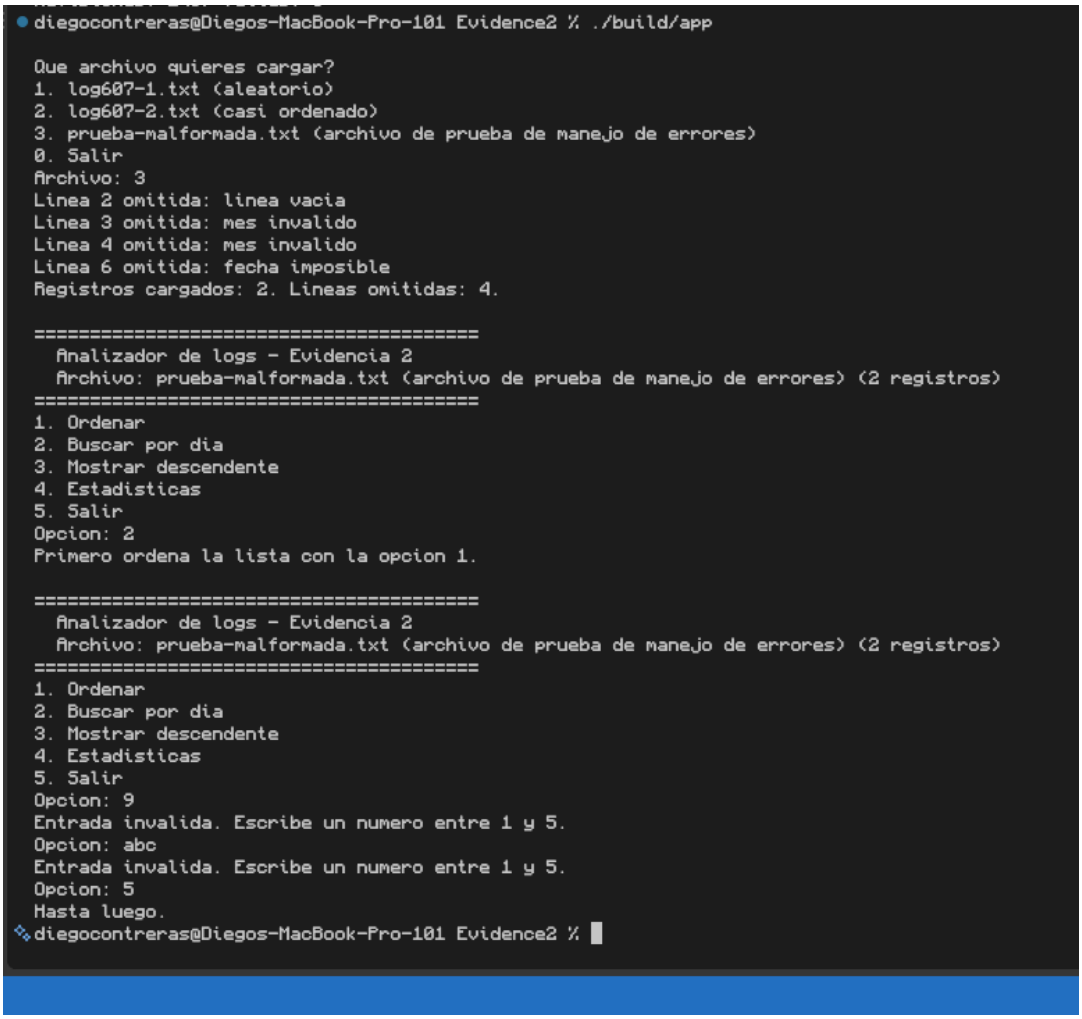


Figura 12. Líneas inválidas en prueba-malformada.txt, buscar antes de ordenar y opciones inexistentes.

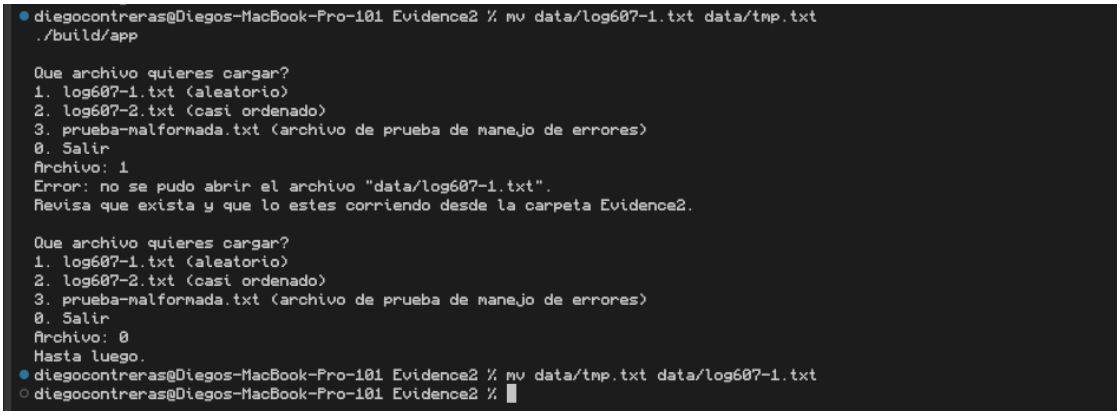


Figura 13. Archivo inexistente.

2. Resultados del ordenamiento

El algoritmo elegido fue Merge Sort, implementado directamente sobre la lista doblemente ligada. No se copian los registros a un arreglo ni se crean nodos nuevos, solo se cambian los apuntadores next y prev. Se eligió Merge Sort porque hace del orden de n log n comparaciones sin importar cómo vengan los datos y porque es estable. Quick Sort en una lista ligada no puede tomar la mediana de tres de forma

barata, porque llegar al nodo de en medio cuesta un recorrido, y con el pivote en un extremo se degrada a n^2 justo con el archivo casi ordenado.

Archivo	Predicción	Tiempo	Comparaciones	n $\log_2$ n	Profundidad máxima	Coincidió	Conservó el orden de llegada
log607-2.txt (casi ordenado)	n log n	1.733 ms	50,624	86,828	13	Sí	Sí
log607-1.txt (aleatorio)	n log n	2.357 ms	78,330	86,828	13	Sí	Sí

En los dos archivos n = 6818 y $n^2/2$ = 23,242,562, así que las comparaciones quedaron muy por debajo de n^2 y cerca de n $\log_2$ n. La profundidad cuenta la primera llamada como 1 y no cuenta las llamadas que reciben una cadena de un solo nodo, porque esas regresan de inmediato sin dividir nada.

Para comprobar que se conservó el orden de llegada de los registros con la misma fecha y hora, la salida se comparó contra la de sort -s, que es un ordenamiento estable de la terminal, usando como llave la fecha y la hora. El resultado fue de 0 diferencias. Esto importa porque los archivos traen 18 pares de registros con la misma fecha y hora, y un algoritmo inestable podría invertir alguno de esos pares. La comparación se hizo con los dos archivos y en los dos dio 0 diferencias. La figura muestra la de log607-1.txt, que es el que quedó en output608.out.



Figura 14. Comparación de output608.out contra un ordenamiento estable: 0 diferencias.

Interpretación

En log607-1.txt, que viene en orden aleatorio, Merge Sort hizo 78,330 comparaciones, un poco menos que n $\log_2$ n. Esto cuadra con la teoría, porque cada mezcla de dos mitades de tamaño k hace a lo mucho 2k menos 1 comparaciones y en datos aleatorios casi siempre se acerca a ese máximo.

En log607-2.txt, que viene casi ordenado, hizo 50,624 comparaciones, alrededor de 35 % menos. La razón está en cómo termina la mezcla. Cuando una de las dos mitades se acaba, lo que sobra de la otra se pega completo sin compararlo. Si los datos ya vienen casi en orden, en muchas mezclas toda la mitad izquierda es menor que la derecha, así que la izquierda se acaba después de k comparaciones y la derecha se engancha entera. El tiempo también bajó, de 2.357 ms a 1.733 ms. Aun así la diferencia es pequeña comparada con algoritmos como insertion sort, que en la Evidencia 1 cambiaba más de cien veces entre un archivo y otro. Merge Sort sigue dividiendo y mezclando lo mismo sin importar el orden de entrada.

La profundidad fue de 13 en los dos archivos. Eso pasa porque Merge Sort siempre parte la lista a la mitad sin ver los datos, así que la profundidad solo depende de n. Como 2^{12} = 4,096 es menor que 6,818 y 2^{13} = 8,192 es mayor, se necesitan 13 niveles de división para llegar a cadenas de un nodo.

3. Prompts utilizados

Usé Claude de dos formas. En el chat lo usé para desglosar las instrucciones del profesor en un plan de once fases, de la 0 a la 10, donde cada fase dice qué reusar de mis actividades anteriores y cómo

comprobar que quedó bien. Y luego implemente cada fase codeando con ayuda de copilot. Implemente las mismas reglas de codigo de la entrega pasada.

El primer prompt clave fue el de las reglas, que escribí antes de empezar:

> *"Vamos a trabajar sobre evidence 2 con las reglas que te voy a poner: no escribir codigo sin mi aprobacion ni hacer commits o cambios sin que yo los halla visto o aprobado."*

Lo usé tal cual y fue lo que más cambió la forma de trabajar respecto a la Evidencia 1. En la primera entrega la herramienta organizaba todos los commits sobre lo que sea que yo acababa de terminar de trabajar. Aquí cada parte yo hacia cambios, compilaba y hacía el commit. Por eso el historial muestra un commit por pieza (stack y queue, lista doble, pruebas, registro, carga, menú, merge sort y búsquedas) en vez de un bloque al final.

El segundo fue la fase del registro, que pedía saber por qué se rechazaba cada línea:

> *"Ayudame con parsearLinea para que regrese el motivo del error, por ejemplo con la firma bool parsearLinea(const std::string& linea, Registro& reg, std::string& motivo). Motivos posibles: linea vacia, mes invalido, fecha imposible, hora invalida, falta la IP, falta la descripcion."*

Con eso el nuevo parsearLinea que separa la línea en sus campos y revisa las partes en orden hasta encontrar la primera que falla. Para no repetir la validación de la fecha, al final reusa fechaAClave de la Evidencia 1. Lo que se agregó fuera del prompt fue una revisión de que la IP tenga solo dígitos y puntos. Sin ella, una línea como Oct 02 2024 23:04:24 Social engineering tomaba la palabra Social como si fuera la IP y aceptaba la línea como válida. Lo comprobé con prueba-malformada.txt: salen los mismos 2 registros válidos que en la Evidencia 1 y las 4 líneas reportadas, ahora cada una con su motivo.

El tercero fue la fase del ordenamiento, la más difícil de la entrega:

> *"En la fase 8. Merge Sort sobre la lista doble. […] ocupo saber como enganchar nodos existentes sin crear nuevos. Al enganchar cada nodo, actualiza su prev. […] Al terminar, recorre hasta el final para volver a fijar tail."*

De ahí salieron dividir, mezclar y mergeSortRec. El cambio principal fue dónde quedaron: dentro de la clase DoublyLinkedList, igual que el sort de mi Actividad 2.2, porque el ordenamiento necesita cambiar head y tail, que son privados. También se agregó a la aplicación una revisión que, después de ordenar, recorre la lista hacia adelante y hacia atrás y avisa si algún prev quedó mal. Para confirmar que el resultado era correcto comparé output608.out contra la salida del Merge Sort de la Evidencia 1 y contra un ordenamiento estable de la terminal, y en los dos casos hubo 0 diferencias.

Además de esos cambios, otras decisiones que tomé fueron escoger con qué archivo se genera output608.out, que fue log607-1.txt, y dejarlo explicado en el README porque las salidas de los dos archivos no son idénticas; y mantener los mensajes de consola sin acentos, igual que en la Evidencia 1. También se encontró que el .gitignore de la raíz del repositorio ignora todos los archivos .out, que es una regla pensada para ejecutables, y que por eso output608.out nunca se iba a subir. Se agregó una excepción solo para ese archivo.

Un error real de la IA y cómo lo detecté

El error más claro fue una comprobación falsa. Para verificar que las estadísticas salen iguales antes y después de ordenar, Claude me ayudo a armar un comando que separaba las dos salidas y las comparaba, y el resultado fue “iguales”. Pero arriba de esa palabra había dos errores de No such file or directory: el paso que separaba las salidas había fallado, así que se compararon dos textos vacíos, y dos textos vacíos siempre son iguales. Se repitió la comparación de otra forma y ahí sí salieron dos bloques de 25 líneas idénticos. Es el mismo tipo de error que la búsqueda binaria sobre datos desordenados de la Evidencia 1: nada truena, solo sale un resultado que parece bueno, y la única forma de notarlo es leer la salida completa y no solo la última línea.

El segundo error venía del plan. Para revisar las estadísticas sugería comparar contra la terminal con sort | uniq -c | sort -rn | head -3. Para el día y el evento eso basta, pero para las IPs el comando muestra tres direcciones con 3 apariciones y esconde que en realidad son 13 empatadas. Si me hubiera guiado solo por ese comando habría pensado que el programa imprimía de más. Se notó porque el programa mostraba 13 direcciones. Al contar en la terminal cuántas IPs tenían exactamente 3 apariciones salieron las mismas 13. El head -3 cortaba justo el caso de empate que la opción 4 tenía que cubrir.

4. Reflexión personal

Lo que aprendí del uso de la IA

Lo más importante que aprendí es que la IA ayuda mucho más cuando se le pone un marco. En la Evidencia 1 no lllevaba tanto orden y revisaba todo al final sin estructuras. En esta trabajé con un plan escrito por fases y con reglas. Eso me obligó a leer cada parte cuando todavía era chica, y por eso puedo explicar el código: cuando llegó el Merge Sort ya conocía la lista doble, porque la había revisado completa en la fase anterior.

Sin la IA me habría tardado bastante más en el menú, la carga y el programa de pruebas, que son código largo y repetitivo. Lo que sí habría hecho igual es dibujar los nodos en papel para el remove y para la mezcla del Merge Sort, porque los casos del head, del tail y del nodo único solo se entienden viendo cómo se mueven los apuntadores.

El lugar de cada estructura

Cada estructura está donde está por la forma en que entra y sale la información. La Fila es el buffer de lectura porque las líneas tienen que procesarse en el mismo orden en que vienen en el archivo: la primera que entra es la primera que sale. No es un detalle, porque de ese orden depende que el Merge Sort, que es estable, deje los 18 pares con la misma fecha en su orden original. La lista doble guarda los registros porque permite agregar al final en O(1) gracias a tail, recorrer en los dos sentidos y ordenar cambiando solo apuntadores, sin copiar ningún registro. La lista simple es la base de la Pila y la Fila, y necesitaba tail para que formarse al final de la Fila también fuera O(1).

La Pila de la opción 3 no era estrictamente necesaria. Con prev se puede llegar al último registro del día y recorrer hacia atrás hasta el primero, sin memoria extra. La ventaja de la Pila es que el código se lee más fácil: se recorre igual que en la opción 2 y el orden al revés sale solo al sacar los elementos, y además es lo que pide la instrucción. La desventaja es que usa memoria extra por cada registro encontrado. En esta aplicación es poco, porque un día tiene a lo mucho 21 registros y la Pila guarda

apuntadores, no copias. Con prev se ahorra esa memoria, pero el recorrido es más delicado: hay que ubicar el último registro del día sin pasarse y luego regresar, y es más fácil equivocarse en los bordes.

Por qué no hay búsqueda binaria en una lista ligada

La búsqueda binaria funciona porque en un arreglo se puede saltar al elemento de en medio en un solo paso. En una lista ligada no se puede. Para llegar al nodo de en medio hay que caminar n/2 nodos desde el head, en la siguiente mitad otros n/4, y así. Sumando, solo llegar a los puntos de en medio cuesta casi n pasos, así que se pierde la ventaja y queda igual o peor que recorrer la lista una vez, que es lo que hace la opción 2 con su corte temprano.

Para hacerla posible habría dos caminos. El primero es copiar los registros, o apuntadores a ellos, a un arreglo. Eso cuesta O(n) en tiempo y en memoria cada vez que cambia la lista, y además lo prohíbe la regla de que los registros solo vivan en la lista doble. El segundo es usar otra estructura pensada para buscar, como una skip list, que agrega a cada nodo apuntadores que se saltan varios nodos a la vez, o un árbol de búsqueda.

Lo que dicen las mediciones

En log607-1.txt, que viene en orden aleatorio, el Merge Sort hizo 78,330 comparaciones en 2.357 ms. En log607-2.txt, que viene casi ordenado, hizo 50,624 en 1.733 ms. Las dos quedaron cerca de n $\log_2$ n, que es 86,828, y muy lejos de $n^2/2$, que serían más de 23 millones. La diferencia entre archivos viene de cómo termina cada mezcla: cuando una mitad se acaba, la otra se pega completa sin compararla, y con datos casi ordenados eso pasa muy pronto. Aun así el cambio es chico comparado con la Evidencia 1, donde insertion sort cambiaba más de cien veces de un archivo a otro. El Merge Sort divide y mezcla igual sin importar cómo vengan los datos, y eso es lo que lo hace predecible.

La profundidad de 13 es el número máximo de llamadas de mergeSortRec abiertas al mismo tiempo, una encima de otra en la pila de llamadas del programa. Cada nivel parte la lista a la mitad, así que la profundidad crece como $\log_2$ n: con 6,818 registros son 13, y con un millón serían unas 20. Por eso no hay riesgo de desbordar la pila de llamadas. Con un Quick Sort que se degrada, como la variante con pivote al final que probé en la Evidencia 1 con el archivo casi ordenado, cada llamada quita un solo elemento y la profundidad puede acercarse a n, que aquí serían miles de llamadas apiladas. Es otra razón, además de la estabilidad, por la que el Merge Sort era la mejor opción para la lista.